第 6 章練習解答　CPU 排程

題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS06-E01　非搶占排程計算

FCFS：P1[0,8)、P2[8,12)、P3[12,14)。SJF：P3[0,2)、P2[2,6)、P1[6,14)。以下固定依工作編號排列，避免把執行次序誤當成答案次序。

工作	FCFS完成／等待／周轉	SJF完成／等待／周轉
P1	8／0／8	14／6／14
P2	12／8／12	6／2／6
P3	14／12／14	2／0／2

FCFS 平均等待20/3，SJF 為8/3，相差4；平均周轉分別34/3與22/3。兩政策皆在14完成全部工作，但分配給個別工作的等待不同。

最短平均等待的交換論證要求同時抵達、實際需求已知、無 I/O及額外切換成本；估計需求若排錯順序，不能保證同一最佳值。

OS06-E02　Round Robin

q=2：P1[0,2)、P2[2,4)、P3[4,6)、P1[6,8)、P3[8,9)、P1[9,10)。

工作	完成	等待＝完成−需求	首次 CPU 回應
P1	10	5	0
P2	4	2	2
P3	9	6	4

平均等待13/3。需求總和5+2+3=10，時間表沒有 idle，也沒有把切換成本算進去。

q=1：每個區間長1，依序執行下表。

區間起點	0	1	2	3	4	5	6	7	8	9
工作	P1	P2	P3	P1	P2	P3	P1	P3	P1	P1

工作	完成	等待	首次 CPU 回應
P1	10	5	0
P2	5	3	1
P3	8	5	2

平均等待仍13/3，但 P2 等待增加1、P3減少1，兩者首次回應都更早。時間片邊界由5增至9個，真正跨工作切換由5增至8次；最後兩片同為P1，不必為時間片結束而切換工作。

OS06-E03　優先權與 starvation

1. 反例：低優先權 L 在0就緒，每個整數k都有高優先權 H_k 到達，恰需1 ms。若高者皆可搶占，H_k 連續占用[k,k+1)，L永遠沒有CPU；這才是無限等待的構造，觀察一次長延遲並不足夠。
2. aging 必須有明確規則。例如優先權0至10、數字大者優先；L從0起每等2 ms提升1級，到20 ms達到最高級。同級採FIFO，新到者不能插隊，已選工作每次最多占CPU B ms，且不額外付切換成本。
3. 若提升時已有q個ready工作在前，另有一個running工作最多剩B，L等候上界為20+(q+1)B ms。要在接納前承諾共同上界，還需q≤N的負載限制，才得到20+(N+1)B。沒有有限前方數量、服務上界與公平同級規則，單寫「會aging」不能推出數值界。

OS06-E04　HRTICK CFS 逐事件推演

1. n=2≤8，所以 period=6 ms；等權重時 ideal_runtime=6×1024/2048=3 ms。nice=0 的 `calc_delta_fair` 不縮放，因此每執行 1 ms，vruntime 增加 1 ms。

2. 以下為處理事件、選出下一個 current 後的狀態。ready 不含 current；min_vruntime 不倒退。

t	current	其他 ready（工作@vruntime）	min_vruntime
0	P1	P2@0	0
3	P2	P1@3	0
6	P1	P2@3	3
9	P3	P2@3、P1@6	3
11	P2	P1@6	3
12	P1	空	6
14	空	空	8

相同 vruntime 依題設入樹次序；t=8 不另加切換。

3. Gantt 為 P1[0,3)、P2[3,6)、P1[6,9)、P3[9,11)、P2[11,12)、P1[12,14)。完成 P1/P2/P3=14/12/11。t=8 沒有 HRTICK、週期 tick、completion 或 wakeup，所以不切換。
4. P3 的門檻為 `max(舊 vruntime, min_vruntime-3)=max(0,3-3)=0`。喚醒比較在 t=9 做，P1 的 vruntime 已到 6；差值 6 ms 嚴格大於 1 ms，所以 P3 在 t=9 立即取得 CPU，不是在 t=8 預先切換。

OS06-E05　週期 tick 的嚴格大於

1. 從 t=0 起的 delta_exec 在 t=3 為 3、t=4 為 4。v3.2 的第一條件是 `delta_exec > ideal_runtime`；3>3 為 false，4>3 才成立。
2. P1 跑 [0,4)，P2 跑 [4,8) 並正好完成；P1 在 [8,9) 跑 1 ms。P3 在 t=9 放回 vruntime=2，與 P1 vruntime=5 相差 3>1，於是 [9,11) 搶占；P1 在 [11,14) 完成。completion=14/8/11。
3. ideal_runtime 是排程器允許的 slice 上限，不是硬體 timer 保證。只有 HRTICK 的 queued tick 會在精確 slice 邊界直接重排；一般 tick 只在解析度允許的檢查點用嚴格大於判斷。
4. t=8.5 發生 wake preemption，舊 CFS HRTICK 的 P1[6,9) 先改成 P1[6,8.5)，接著 P3[8.5,10.5)、P2[10.5,11.5)、P1[11.5,14)；全部 completion 改為 P1=14、P2=11.5、P3=10.5。

OS06-E06　CPU 公平與同步語意分離

1. 總 weight=1359；A 的 slice=6×1024/1359≈4.521 ms，B 的 slice≈1.479 ms。CPU share 分別為約 75.35% 與 24.65%，不是固定相同 quantum。
2. 不能。若 A、B 各執行 ordinary read-modify-write `counter++`，同一個 CFS runqueue 仍可能讓兩者的 load/add/store 交錯，最後少於 2。排程公平只約束 CPU 分配，不保證複合操作的原子性；若以 C/C++ 實作，未同步的衝突非原子存取還會形成 data race，行為未定義。
3. B 以 atomic release store 發布 ready，A 用讀到該次發布值的 atomic acquire load 觀察 ready；或使用具有等價 ordering 的 mutex/condition-variable 協定。只有 ready 的數值變 1，不能證明 buffer 可見。
4. mutex 提供同一 critical section 的 mutual exclusion；若 API 明確規定 lock acquire 與 unlock release 的 memory-ordering，也提供對應可見性。單說「有 mutex」仍要指定等待、釋放、錯誤路徑與 publication 是否在同一保護範圍。
5. memory barrier 只限制記憶體操作的排序或可見性，不建立兩 threads 進入區段的互斥；atomic operation 讓指定位置不可分割，並可依 release/acquire/seq_cst contract 發布另一位置的資料。mutex 以 ownership 保護整段 critical section，成功 lock/unlock 通常同時建立互斥與 happens-before。binary semaphore 初值 1 可讓一個 wait 取得 permit，因而序列化區段，但沒有 owner；counting semaphore 控制 permit 數量，是否提供 memory ordering 取決於 API。

OS06-E07　詳解

1・界與干擾。 甲組 $U=2/5+4/8=0.9$，大於 $2(\sqrt{2}-1)\approx0.8284$；充分測試無法判定，不是失敗證明。迭代為 $4\to6\to8\to8$：$4+\lceil4/5\rceil2=6$，$4+\lceil6/5\rceil2=8$，$4+\lceil8/5\rceil2=8$。$J_{2,0}$ 恰在期限8完成；最高優先權的 $J_{1,0}$ 於2完成。

2・RM 首段。 $[0,2)$ $J_{1,0}$；$[2,5)$ $J_{2,0}$ 做3、剩1；$[5,7)$ $J_{1,1}$；$[7,8)$ $J_{2,0}$ 完成；$[8,10)$ $J_{2,1}$ 做2、剩2。$J_{1,0}/J_{1,1}$ 完成2/7、期限5/10；$J_{2,0}$ 完成8、期限8。這只核對首段，不靠一段圖宣稱所有未來 RM job 都準時。

3・EDF 逐事件。 括號為（剩餘，絕對期限），表中選擇發生在該時刻的完成與到達處理之後。

t	ready jobs	執行至下一事件
0	$J_{1,0}$(2,5)、$J_{2,0}$(4,8)	$J_{1,0}$：0–2
2	$J_{2,0}$(4,8)	$J_{2,0}$：2–5
5	$J_{2,0}$(1,8)、$J_{1,1}$(2,10)	$J_{2,0}$：5–6，續作
6	$J_{1,1}$(2,10)	$J_{1,1}$：6–8
8	$J_{2,1}$(4,16)	$J_{2,1}$：8–10
10	$J_{2,1}$(2,16)、$J_{1,2}$(2,15)	$J_{1,2}$：10–12，搶占
12	$J_{2,1}$(2,16)	$J_{2,1}$：12–14
14	空	idle：14–15
15	$J_{1,3}$(2,20)	$J_{1,3}$：15–16，尚餘1

在16之前到期的 jobs：$J_{1,0}$/$J_{1,1}$/$J_{1,2}$ 完成2/8/12，期限5/10/15；$J_{2,0}$/$J_{2,1}$ 完成6/14，期限8/16，全部通過。t=8 先完成 $J_{1,1}$ 再加入 $J_{2,1}$；t=16 新 $J_{2,2}$ 到達，但不改變舊 $J_{2,1}$ 已完成的事實。U=0.9≤1 且 D=T，理想 EDF 充要條件才保證無限序列；不能拿 [0,16) 代替一般證明。

4・充分不是必要。 乙組 U=1。RM 的 [0,1) $J_{1,0}$、[1,2) $J_{2,0}$、[2,3) $J_{1,1}$、[3,4) $J_{2,0}$，完成1/3/4，期限2/4/4。t=4 無積欠且同步釋放，狀態重現，可每4單位重複。故 U>0.8284 仍可排程。

5・短期限與模型。 丙組 U=0.4，但 L=3 時各 task 的 dbf=max(0,⌊(3−3)/10⌋+1)・2=2，總需求 4>3；至少一筆超時，與政策無關。甲組 D=T 可用前述 EDF 定理；丙組 D<T 不符前提。加入鎖阻塞後 CPU 有時無法執行最早到期的 ready 工作，原忙碌區間論證不再直接適用，需另給阻塞上界及資源協定的分析；priority inheritance 的名稱本身不是新保證。

OS06-E08　到達不一定引起搶占

SRTF：P1[0,2)、P2[2,5)、P3[5,6)、P1[6,11)。t=2 時 P1剩5，P2需3，故搶占；t=4 時 P2與P3皆剩1，依保留current規則不切換。

工作	完成 f	周轉 f−a	等待 f−a−b	首次 CPU 回應
P1	11	11	4	0
P2	5	3	0	0
P3	6	2	1	1

平均等待(4+0+1)/3=5/3。P2 連續執行的區間跨過 t=4 到達事件，並不違反搶占式政策。

非搶占 SJF：P1[0,7)、P3[7,8)、P2[8,11)。P1 開始時只有它可執行，之後不因新到工作中斷。

工作	完成 f	周轉 f−a	等待 f−a−b	首次 CPU 回應
P1	7	7	0	0
P2	11	9	6	6
P3	8	4	3	3

平均等待3。SRTF 在此較低，但 P1 完成較晚；個別結果與平均結果要分開判讀。

OS06-E09　MLFQ 被打斷的時間額度

A 在 t=2 耗盡 Q0 額度，降到 Q1；B 在 Q0 完成後，A 才開始用 Q1 的4單位額度。

區間	工作／層級	工作剩餘需求	該層剩餘額度／結果
0–2	A／Q0	5	0，降 Q1
2–4	B／Q0	0	完成
4–5	A／Q1	4	3，被 C 搶占
5–6	C／Q0	0	完成
6–9	A／Q1	1	0，降 Q2
9–10	A／Q2	0	完成

工作	完成	等待＝完成−到達−需求	首次 CPU 回應
A	10	3	0
B	4	1	1
C	6	0	0

t=6 若把 A 的額度重設為4，它可在 Q1 連跑6–10並完成，不再進 Q2。雖然本例完成時間恰巧相同，層級與額度已不同；若此時另有 Q1 工作，後續選取也可能改變。不能用相同完成時間掩蓋政策差異。

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.
Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study